

HELM: Compiler-Guided Heterogeneous Execution for Large Language Models on Memory-Constrained Systems

Atharva Khedkar*, Graduate Student Member, IEEE, Shashwat Pandey*, and Aviral Shrivastava, Senior Member, IEEE

Abstract—Large language model (LLM) execution on commodity hardware with a single discrete GPU remains challenging because model weights, the KV cache, or both can exceed available VRAM. Existing inference backends either require all weights to fit in GPU memory or stream layer weights across the PCIe bus during execution, incurring repeated data transfers that severely limit decode throughput.

We present HELM – Compiler-Guided Heterogeneous Execution for LLMs on Memory-Constrained Systems. HELM partitions model layers across CPU and GPU at compile time. HELM uses hardware-calibrated roofline models to search over all feasible contiguous layer-to-device assignments and to select the partition that minimizes per-token decode latency. The plan is compiled into static per-device execution graphs. This eliminates per-token weight movement and reduces cross-device traffic to a single activation transfer per decode step. HELM further includes a paged KV-cache manager that offloads inactive cache pages to CPU memory and an asynchronous streaming-attention kernel. Together they extend the usable context window well beyond GPU VRAM capacity.

We compare HELM against vLLM, Hugging Face Accelerate, and DeepSpeed ZeRO-Inference on models ranging from 4B to 32B parameters. In memory-constrained systems where existing backends fail with out-of-memory (OOM) errors, HELM sustains inference by partitioning layers across devices. Where competing backends remain feasible, HELM achieves up to $10.4\times$ higher decode throughput, with a geometric-mean speedup of $6.2\times$ over the strongest feasible baseline, when weights exceed VRAM, while extending the maximum output length by up to $256\times$ (geometric mean $5.9\times$ on feasible Qwen3 family). HELM is available at <https://github.com/MPSLab-ASU/HELM>.

Index Terms—Large language models, compilers, heterogeneous computing, LLM inference, transformers, roofline model, KV-cache offloading, runtime

I. INTRODUCTION

LARGE language models (LLMs) have become central to modern computing workflows, supporting tasks including web search, code generation, scientific reasoning, and interactive assistants. Driven by privacy and security concerns, as well as the demand for offline, on-device availability, both the tech industry and end users are pushing toward local LLM execution through tools such as LM Studio [1] and Ollama [2], alongside frameworks such as OpenClaw [3] for building personalized agents that interface with locally hosted

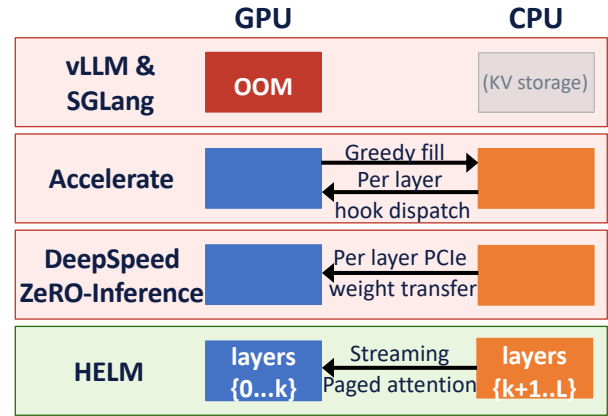


Fig. 1. Comparison of HELM with state-of-the-art inference engines. Default vLLM [6] and SGLang [7] require model weights to fit in VRAM and fail with OOM errors when they do not. vLLM’s CPU-offload path uses host memory as backing storage for GPU execution, so offloaded weights are still accessed or loaded during forward passes rather than executed as stationary CPU-resident layers. Accelerate [8] and DeepSpeed [9] similarly incur significant PCIe overhead by moving weights during execution. HELM eliminates redundant weight movement. By statically partitioning the model into per-device subgraphs, HELM minimizes latency through hardware-aware layer splitting, reduces cross-device traffic to a single activation transfer per token, and manages memory-intensive context via asynchronous KV-cache offloading.

models. Larger models consistently deliver better performance. For complex reasoning and problem-solving, at least a 7-billion-parameter model (7B+) is required [4], [5]. However, the memory requirements of larger LLMs are fundamentally incompatible with consumer hardware. For example, a half-precision (fp16) 7B-parameter model requires ≈ 14 GB of GPU memory; a 32B-parameter model requires ≈ 64 GB. The overwhelming majority of consumer and prosumer GPUs provide only 8–24 GB of VRAM.

Local cost-constrained inference and embedded deployment rely on commodity hardware: edge accelerators and laptops with a modest discrete GPU, where GPU VRAM is limited, host DRAM is a separate memory pool, and the two devices communicate over PCIe. In these settings, limited VRAM, heterogeneous CPU–GPU configurations, and the lack of a multi-GPU node make efficient LLM inference particularly challenging. Enabling that inference still expands accessibility and reduces dependence on specialized hardware. Existing systems address this problem in ways that leave substantial performance on the table. Engines such as vLLM [6] and SGLang [7] are designed for multi-request LLM serving, and in their default GPU-resident configurations require full model weights to fit in VRAM. vLLM also supports CPU weight

This work was supported in part by the National Science Foundation under Grant ACED 2436016 and in part by the Semiconductor Research Corporation (SRC) under Project 3154.

A. Khedkar and A. Shrivastava are with Arizona State University, Tempe, AZ, USA (e-mail: atharva.khedkar@asu.edu; aviral.shrivastava@asu.edu). S. Pandey was with Arizona State University, Tempe, AZ, USA, when this work was performed and is now with AGI, Inc. (e-mail: spande90@asu.edu).

*Equal contribution.

offload, but this treats host memory as backing storage for GPU execution: offloaded weights are accessed or loaded during each forward pass rather than executed as stationary CPU-resident transformer layers. SGLang can offload KV cache pages to host memory, but still requires model weights to fit in VRAM and does not execute dense transformer layers on the CPU. Hugging Face Accelerate [8] supports partial CPU offloading via a greedy strategy that first fills the GPU, then spills the remaining layers to the CPU with no performance model. DeepSpeed ZeRO-Inference [9] streams model weights from CPU to GPU one layer at a time, incurring $O(L)$ PCIe transfers regardless of how little computation each transfer enables, where L is the number of transformer layers. Fig. 1 illustrates these failure modes and contrasts them with our proposed approach. We observe that the layer-to-device assignment problem is fundamentally a *static* decision: the model architecture, weight sizes, and hardware capabilities (DRAM bandwidth, GPU memory, PCIe throughput) are all known before the first token is generated and do not change across requests. This makes it naturally suited to a compile-time formulation. A compiler can afford to exhaustively evaluate every feasible partition point and select the optimal partition—a search best deferred to compile time, where one-time hardware profiling is available. Once the partition is fixed, we place each weight on its target device exactly once and compile the plan into static per-device graphs, eliminating the per-token weight streaming, runtime placement heuristics, and Python-level dispatch overhead that together penalize existing approaches.

We present **HELM – Compiler-Guided Heterogeneous Execution for Large Language Models** – a compiler and runtime system that treats single-node CPU–GPU inference as a compile-time optimization problem. As shown in Fig. 2, HELM profiles the target hardware and builds a roofline cost model that evaluates every feasible contiguous layer split and selects the partition that minimizes per-token decode latency. The compiler lowers this partition into per-device static execution graphs, where model weights are placed once and never transferred during token generation. At runtime, each layer executes on the device that holds its weights. CPU-resident layers use an AVX2+F16C GEMV kernel, and only a single hidden-state activation tensor crosses the PCIe boundary per token, replacing the $O(L)$ weight transfers of layer-streaming approaches. HELM also provides a paged KV-cache manager that evicts cold pages to CPU RAM and streams them back on demand with streaming paged attention, decoupling context length from GPU memory.

This paper makes the following contributions:

- 1) **Compiler-driven heterogeneous LLM inference.** We formulate CPU–GPU LLM inference as a compile-time optimization problem and introduce a compiler pipeline that generates static per-device execution graphs, eliminating per-token weight transfers (§IV-A).
- 2) **Hardware-calibrated roofline partitioning.** We develop a transformer-block-level cost model that incorporates measured FLOP/s, memory bandwidth, and PCIe transfer costs; we use it to exhaustively search all contiguous CPU–GPU splits in $O(L)$ time and select

the minimum-cost partition under the roofline model (§IV-A3).

- 3) **AVX2+F16C GEMV kernel for CPU-resident layers.** We implement a JIT-compiled C++ kernel using F16C and AVX2 FMA that achieves near-peak memory bandwidth for decode on the CPU, replacing PyTorch’s unoptimized fp16 linear path (§IV-A5).
- 4) **Paged KV-cache with streaming attention.** We design a KV-cache manager that evicts inactive pages to host DRAM and computes attention with a streaming online-softmax kernel, bounding GPU-resident KV so context length can grow beyond residual VRAM (§IV-B).

HELM requires no manual configuration: all parameters, including the KV eviction threshold, use validated defaults. Given a model and target hardware, a single command installs dependencies and launches inference with automatic hardware profiling, partition selection, and graph compilation. Our evaluation across three GPU configurations and ten models from five architecture families (4B–32B parameters) demonstrates that HELM sustains inference across memory-constrained configurations where GPU-resident backends fail with out-of-memory errors. On memory-constrained configurations where competing fp16 backends remain feasible, HELM achieves up to $10.4\times$ higher decode throughput, with a geometric-mean speedup of $6.2\times$ over the strongest feasible baseline. By decoupling context capacity from GPU VRAM through its paged KV-cache manager, HELM extends the maximum supported output length by a geometric mean of $5.9\times$, and by up to $256\times$ in the most constrained settings, on feasible Qwen3 pairs in Fig. 5.

II. BACKGROUND

A large language model is a stack of L *transformer blocks*, bookended by a token-embedding layer and an output projection. Each block pairs a multi-head attention sub-layer with a feed-forward network (FFN); each sub-layer is preceded by RMSNorm and wrapped in a residual connection. Most recent open-weight decoder-only models use *grouped-query attention* (GQA), where several query heads share a smaller number of key/value heads, reducing the per-token KV storage (the n_{kv} term below). The FFN typically holds three gated-linear matrices (gate, up, down; SwiGLU in Qwen3/LLaMa, GEGLU in Gemma-2) and dominates the per-block weight bytes (roughly 78% in Qwen3-8B). Each block maps a hidden state of width H to another of width H , so a contiguous run of blocks can execute on one device and, at decode, a single H -vector crosses the boundary; this makes layer-level partitioning natural.

Inference has two phases. *Prefill* processes all S input tokens together as $(S, H) \times (H, H)$ matmuls: high arithmetic intensity, *compute-bound*. *Decode* generates one token per step as $(1, H) \times (H, H)$ matrix-vector products at an arithmetic intensity near 1 FLOP/byte, far below any modern GPU or CPU compute-to-bandwidth ratio, so decode is *memory-bandwidth-bound*. Prefill is a one-time cost proportional to input length; decode repeats per output token, so we minimize decode latency. Because decode is bandwidth-bound, a device’s decode

rate tracks its memory bandwidth, not its peak FLOPs: a CPU at 45 GB/s is roughly $5\times$ slower than a mid-range GPU at 218 GB/s, so placing part of the model on the CPU and never moving its weights makes the CPU contribute real work rather than sit idle.

A second, weight-independent source of memory pressure is the *KV cache*, which stores the key and value vectors of every past token to avoid recomputing attention each step. For L layers, n_{kv} KV heads, and head dimension d_h in half precision, it grows by

$$2 \times L \times n_{kv} \times d_h \times 2 \text{ bytes per token.} \quad (1)$$

Even when weights fit in VRAM, the cache exhausts the residual memory at moderate context lengths: on 8–24 GB consumer GPUs where weights already consume most of the budget, the few hundred megabytes left can cap usable context at a few hundred tokens. This is the problem our paged KV-cache manager addresses.

III. RELATED WORK

The challenge of deploying large language models on memory-constrained hardware has motivated a large body of work. Prior systems differ along two dimensions: what they offload (model weights or the KV cache) and how they use the CPU, from a passive weight store to an active compute participant.

A. Weight Offloading Approaches for Large Models

When model weights exceed GPU VRAM, most engines stream each weight tensor to the GPU just before it is needed. Hugging Face Accelerate [8] implements `device_map=auto`, which greedily fills GPU VRAM and then spills the remaining layers to the CPU. CPU-assigned layers are moved to the GPU just before their forward pass and freed immediately after via dispatch hooks, introducing Python-level synchronization at each layer boundary. vLLM [6] supports partial weight offloading via `cpu-offload-gb`, which uses host memory as backing storage: the offloaded subset is still accessed or loaded for GPU execution during each forward pass, so the CPU is an offload tier rather than an active compute stage.

DeepSpeed ZeRO-Inference [9] pins model weights in CPU RAM or NVMe, streams each layer to the GPU via DMA, computes it, and evicts it, prefetching the next layer to overlap transfer with execution. This incurs $O(L)$ PCIe transfers per generated token, where L is the number of transformer layers, so single-request decode is communication-bound unless batches are large enough to hide the transfers—a condition that generally does not hold for local single-user inference. FlexGen [10] extends sequential offloading to a three-tier GPU/CPU/NVMe hierarchy and formulates tensor placement as a linear program over weights, activations, and KV cache, targeting throughput at batch sizes of ~ 100 – 1000 . Its public implementation is built around OPT and does not ship support for recent families such as LLaMA and Qwen.

llama.cpp [11] is the exception among widely used engines: it permanently assigns the last N transformer blocks to

VRAM and executes the remainder on the CPU with SIMD kernels, so those weights remain stationary. N is a static `n-gpu-layers` parameter that the user must set per model and card. llama.cpp requires GGUF and does not natively load safetensors.

B. KV Cache Offloading for Long-Context Inference

Memory pressure also arises when the model fits in VRAM but the KV cache does not, as with long sequences or large concurrent batches. FastDecode [12] and NEO [13] offload KV state and the associated attention computation to the CPU, freeing GPU VRAM for additional concurrent requests. LayerKV [14] performs layer-wise KV offloading with overlapped transfer, targeting serving latency rather than weight overflow. HeadInfer [15] offloads KV at head granularity, keeping only one or a few heads GPU-resident at a time while still computing all heads. LMCache [16] stores extracted KV caches in CPU memory, local disk, or remote storage to reuse shared prefixes across queries. vLLM’s PagedAttention [6] pages the GPU KV cache to cut fragmentation for continuous batching; its LMCache integration likewise targets prefix reuse, not stationary CPU execution of overflow weights. These systems assume the model itself fits in VRAM and address KV pressure, so they are orthogonal to HELM’s weight-partitioning problem.

C. CPU–GPU Parallel Execution

A more recent line of work uses the CPU and GPU as coequal compute devices on each forward pass. HeteGen [17] applies heterogeneous tensor parallelism to linear modules, splitting each projection so the CPU computes a host-resident fraction while the GPU’s share is transferred over PCIe, overlapped with CPU work. TwinPilots [18] pins the first layers in VRAM, keeps the rest in CPU RAM, and schedules unpinned-layer operations so CPU compute hides weight prefetch, reporting up to $3.39\times$ higher throughput than FlexGen. NEO [13] keeps weights on the GPU and offloads decode attention and KV for a subset of requests to the CPU, overlapping that work with GPU linear ops; it therefore applies only when the model already fits in VRAM. Pipelined sharding [19] profiles kernels at install time and, for each token-count tier, selects among a GPU-only plan, a static sub-layer split that keeps unpinned weights stationary, and a dynamic plan that still streams some weights overlapped with compute. Learned placement methods such as GiPH [20] assign DAG tasks to heterogeneous devices with a reinforcement-learning policy, but they optimize makespan on dynamic clusters rather than decode latency on a memory-constrained CPU–GPU node. HELM instead assigns entire transformer blocks to one device at compile time, keeps those weights stationary, and moves only the activation tensor at a single layer boundary.

D. Sparsity-Aware Neuron-Level Offloading

PowerInfer [21] and Q-Infer [22] exploit power-law activation sparsity: hot neurons activate for most inputs, while

cold neurons activate infrequently. PowerInfer solves an offline integer linear program to preload hot neurons onto the GPU and route cold neurons to the CPU, achieving up to $11.69\times$ speedup over llama.cpp. Q-Infer extends this with dynamic asynchronous neuron loading and KV cache-aware scheduling. These systems require ReLU-family or specially retrained sparse activations. Dense models such as LLaMA-3, Qwen3, and Mistral use SwiGLU, which does not provide the exact-zero sparsity those pipelines rely on, so neuron-level offloading is inapplicable without retraining.

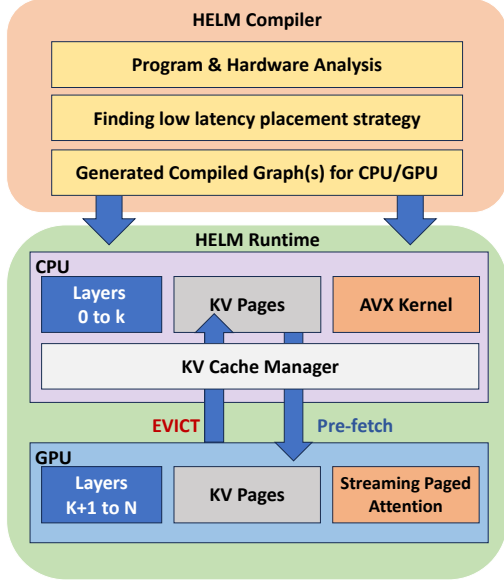


Fig. 2. HELM architecture overview. The HELM compiler analyzes the model graph and hardware profile to determine a low-latency layer placement strategy, emitting separate compiled execution graphs for CPU and GPU. At runtime, the CPU executes layers $0, \dots, k-1$ (CPU stage) with fp16 weights using a custom AVX2 GEMV kernel, while the GPU executes layers $k, \dots, N-1$ (GPU stage) with streaming-paged attention for CPU-resident KV pages. A unified KV-cache manager spans both devices, evicting cold KV pages from GPU to CPU RAM and prefetching them on demand, decoupling context length from GPU memory capacity.

IV. SYSTEM OVERVIEW

Fig. 2 shows the overall HELM architecture. HELM works in two phases: a compile phase that builds a static CPU–GPU plan once per model–hardware pair, and a runtime phase that executes the compiled resident graphs at every decode step.

A. HELM Compiler

When a model’s weights exceed GPU VRAM, HELM’s compiler determines the best way to split the model across CPU and GPU before any inference runs. Placing too many layers on the CPU stalls the GPU; placing too few layers on the CPU causes the GPU to run out of memory. We solve this by finding, at compile time, the split that minimizes decode latency for the given model and hardware, and compiling the result into static, device-specific execution graphs that run unchanged at every decode step.

The compiler works in five stages: (i) *program analysis*, which traces the model’s FX graph and generates HELM’s typed Intermediate Representation (IR), annotating each transformer layer with its weight, activation, and KV-cache costs;

(ii) *hardware profiling*, which microbenchmarks the CPU and GPU to measure actual compute throughput and memory bandwidth; (iii) *partition search*, which checks whether the model fits entirely on the GPU and, if not, evaluates all feasible split points to select the one with the lowest predicted decode latency; (iv) *graph reconstruction*, which splits the FX graph at the chosen boundary into a CPU subgraph and a GPU subgraph, each compiled independently for its target device; and (v) *CPU kernel patching*, which JIT-compiles the AVX2+FP16 GEMV kernel via `cpp_extension.load` and patches it into the CPU subgraph, ensuring CPU linear layers reach the DRAM bandwidth ceiling assumed by the cost model. Fig. 3 shows the complete compilation pipeline.

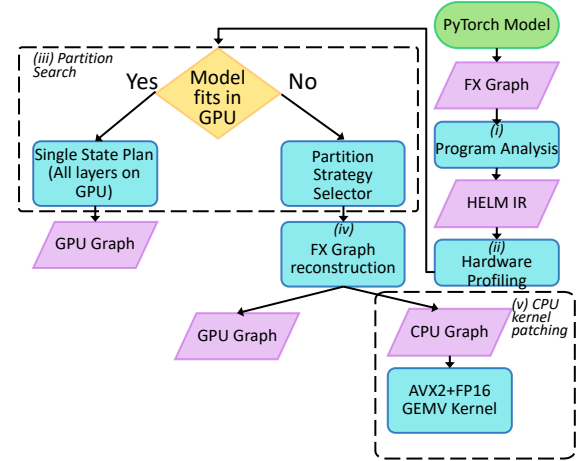


Fig. 3. HELM compiler pipeline. A PyTorch model is traced into the HELMGraph IR, annotated with per-layer costs, and combined with measured hardware profiles to search for the optimal CPU–GPU partition boundary. The resulting subgraphs are compiled once and executed without modification at runtime.

Compilation is a one-time cost per model–hardware pair: the compiled stage graphs and hardware profiles are persisted to disk and reloaded on subsequent runs, so compilation overhead is incurred only when the model or hardware configuration changes.

1) *Program Analysis*: For partitioning, the compiler must know what each transformer block costs: the weight bytes read per decode token (W_{param}), the activation bytes that would cross the CPU–GPU boundary if a split were placed after that block (W_{act}), and the KV-cache bytes accessed per decode step (W_{kv}). PyTorch’s `torch.fx` tracer produces a model’s compute graph with no information about per-block FLOP counts, inter-block activation sizes, or weight byte estimates, making placement decisions from such an IR difficult.

HELM therefore extends the FX graph IR into a typed intermediate representation (HELMGraph IR). **HybridAnalyzer** annotates this IR in two passes. The first pass executes the graph on a representative input to capture the input and output tensor shapes of every operation. The second pass walks the shape-annotated graph, annotating each transformer block with W_{param} , W_{act} , and W_{kv} . These cost annotations sit at **transformer-block** granularity: each block is a group of one attention module, one MLP, and the associated norms. The annotated blocks are then grouped into **Partition Units**: one unit per transformer block plus an embedding unit and an

output-projection unit (38 partition units total for Qwen3-8B).

The per-block resident-memory traffic per decode step is:

$$C_{\text{block}} = W_{\text{param}} + W_{\text{kv}} \quad (2)$$

where W_{param} is the weight bytes streamed from memory and W_{kv} is the KV-cache bytes read during attention. The boundary activation W_{act} is not charged per block; it is charged once per split through T_{comm} in Eq. 17. A complete list of cost model notations is given in Table I. We now derive each term in turn.

a) W_{param} : *weight bytes per block.*: W_{param} is the dominant cost term under the roofline model during decode. Each transformer block's weight footprint decomposes as:

$$W_{\text{attn}} = \underbrace{(H \cdot (n_q d_h))}_{W_Q} + \underbrace{(H \cdot (n_{\text{kv}} d_h))}_{W_K} + \underbrace{(H \cdot (n_{\text{kv}} d_h))}_{W_V} + \underbrace{(n_q d_h) \cdot H}_{W_O} d_{sz} \quad (3)$$

$$W_{\text{FFN}} = \underbrace{(H \cdot d_{\text{ff}})}_{W_{\text{gate}}} + \underbrace{(H \cdot d_{\text{ff}})}_{W_{\text{up}}} + \underbrace{(d_{\text{ff}} \cdot H)}_{W_{\text{down}}} d_{sz} \quad (4)$$

Here, H is the hidden dimension, n_q is the number of query heads, n_{kv} is the number of key-value heads (with $n_{\text{kv}} \leq n_q$ under Grouped-Query Attention), d_h is the per-head dimension read from the model configuration, and d_{sz} is bytes per element (2 for fp16). In architectures that configure d_h independently, the concatenated query-head width $n_q d_h$ need not equal H (e.g., Qwen3-32B: $64 \cdot 128 = 8192 \neq 5120$). Each transformer layer also contains RMSNorm and QK-norm; however, their weights are negligible relative to the total memory cost per layer. For Qwen3-8B, which contains $L = 36$ transformer blocks ($H=4096$, $n_q=32$, $n_{\text{kv}}=8$, $d_h=128$, $d_{\text{ff}}=12288$, fp16), this gives $W_{\text{attn}} \approx 83.9$ MB and $W_{\text{FFN}} \approx 302.0$ MB, summing to:

$$W_{\text{param}} = W_{\text{attn}} + W_{\text{FFN}} \approx 386 \text{ MB per block} \quad (5)$$

b) W_{act} : *activation bytes at the CPU-GPU boundary.*:

At decode time ($B=1$), this is a single hidden-state vector of dimension H :

$$W_{\text{act}} = B \times H \times 2 \text{ bytes} \quad (6)$$

For Qwen3-8B, this is 8 KB per boundary transfer. This is orders of magnitude smaller than W_{param} , so PCIe transfer of activations is not the bottleneck at $B=1$.

c) W_{kv} : *KV-cache bytes per block.*: Each of the n_{kv} heads stores a key and value vector of length d_h , with current context length c in fp16:

$$W_{\text{kv}} = 2 \times n_{\text{kv}} \times d_h \times 2 \text{ bytes} \times c \quad (7)$$

For our Qwen3-8B example, $W_{\text{kv}} = 4096c$ bytes. At $c=4096$ tokens, this reaches 16 MB per block. Together with the once-per-split activation-transfer term, these quantities characterize the memory and communication costs over which the **Partition Selector** optimizes.

2) *Hardware Profiling*: The IR provides the compiler with per-block cost annotations, but we must calibrate these theoretical costs against actual hardware. The largest divergence is memory bandwidth: the roofline model assumes peak DRAM bandwidth, yet sustained throughput reaches only 60–80% of it due to row-activation overhead and memory controller

contention. Secondary factors include kernel-launch and CPU-dispatch latency, which accumulate across the many small operations within a block, and operating-system scheduling jitter, which intermittently stalls the inference thread. HELM therefore measures hardware capabilities directly via microbenchmarks that run once per process and cache their results. Three benchmarks run at startup:

(i) **DRAM bandwidth**: On the GPU, we measure M with a device-side copy. We swept working-set sizes of 64, 128, 256, and 512 MB on an RTX 3090 (830, 838, 842, and 842 GB/s); bandwidth plateaus by 128 MB, so the copy is DRAM-bound rather than cache-resident. We keep 256 MB as a conservative size for GPUs with larger caches. On the CPU, we stream through 256 separately allocated 2 MB buffers (512 MB resident in total, generating about 1 GB of read-plus-write traffic), sized to exceed the L3 cache and to exercise TLB and prefetch behavior, rather than to measure a cache-resident rate. This value fills the denominator of W_{param}/M in the projection cost term and $W_{\text{kv}}/\hat{M}$ in the attention cost term, where $\hat{M}$ is the effective KV bandwidth, a blend of DRAM and L3-cache bandwidth, depending on how much of the KV fits in the cache.

(ii) **Compute throughput**: We set decode throughput P_{dec} from the DRAM bandwidth in (i) at about 1 FLOP/byte (fp16 GEMV), and we measure prefill throughput P_{pre} with a $(128, H) \times (H, H)$ matrix multiply. P_{dec} completes the roofline form; P_{pre} is unused for the decode split.

(iii) **PCIe bandwidth**: We measure B_{link} with a 64 MB pinned copy in each direction and keep both. Decode uses the host-to-device rate. At single-token decode the activation is a few kilobytes, so $W_{\text{act}}/B_{\text{link}}$ is sub-microsecond and identical on every mixed plan.

We also read C_{L3} from `cpuinfo` and set $B_{L3} = 3M$. The profiler stores one *DeviceProfile* per device,

$$\mathcal{D} = (P_{\text{dec}}, P_{\text{pre}}, M, C_{L3}, B_{L3}) \quad (8)$$

The $\mathcal{D}$ is the hardware input to that stage's T_{proj} and T_{attn} . The profiler also stores one *LinkProfile* per copy direction, $\mathcal{L}_{(s,d)} = B_{\text{link}}^{(s \rightarrow d)}$, which is the hardware input to the bandwidth term of T_{comm} . For our running example, the microbenchmarks yield $M_{\text{CPU}} = 45$ GB/s (host DDR4), $M_{\text{GPU}} = 218$ GB/s (RTX 4060 VRAM), and $B_{\text{link}} = 16$ GB/s (PCIe). Together with the per-block costs from §IV-A1, these values supply the hardware inputs to the cost model.

3) *Partition Search*: The compiler now has per-block costs from the IR and hardware capabilities from the profiler. The search intuition is straightforward. Moving blocks from GPU to CPU reduces GPU memory pressure, making more configurations feasible, but increases CPU compute time. Moving blocks the other way reduces CPU time, but eventually renders the plan infeasible when weights no longer fit in VRAM. We find the optimal boundary k by evaluating the total predicted latency for every feasible split and selecting the minimum.

(i) **Total plan cost**: Let L be the total number of transformer layers in the model and let $U = L + 2$ be the number of partition units, including the embedding and output projection. For a split index k , the CPU stage executes units $\{u_i\}_{i < k}$ and the GPU stage executes units $\{u_i\}_{i \geq k}$. T_{comm} is the

TABLE I
NOTATION USED IN THE COST MODEL

Symbol	Unit	Definition
<i>From model config (analytic)</i>		
c	–	Current context length.
H	–	Hidden dimension.
n_q, n_{kv}	–	Query heads, KV heads.
d_h	–	Head dimension (from model config).
d_{ff}	–	MLP intermediate width.
d_{sz}	bytes	Bytes per element (2 for fp16).
W_{param}	bytes	Weight bytes per block per decode step.
W_{act}	bytes	Activation size at layer boundary.
W_{kv}	bytes	KV-cache bytes read per decode step.
<i>From microbenchmarks (measured)</i>		
M	GB/s	Sustained DRAM bandwidth.
P_{pre}	FLOP/s	Prefill compute throughput (full prompt).
B_{link}	GB/s	Sustained one-way PCIe bandwidth (host-to-device for decode).
<i>Queried or estimated (not benchmarked)</i>		
C_{L3}	bytes	L3 cache size (from <code>cpuinfo</code>).
B_{L3}	GB/s	L3 bandwidth ($3 \times M$, estimated).
P_{dec}	FLOP/s	Decode compute throughput (from M at ~ 1 FLOP/byte).
ℓ_{link}	ms	Per-crossing stage-boundary overhead (constant; same on every mixed plan).
<i>Derived</i>		
$\hat{M}$	GB/s	Effective KV bandwidth (L3–DRAM blend).
F_{proj}	FLOPs	FLOPs for all projection/linear layers in the stage, per decode step.
F_{attn}	FLOPs	Attention FLOP count for one stage during one decode step.
T_{proj}	ms	Projection-term decode latency for one stage (max of compute time and parameter-streaming time).
T_{attn}	ms	Attention-term decode latency for one stage (max of compute time and KV-read time).
α	–	KV working-set fraction in L3.
η_c, η_m	–	Compute and memory efficiency coefficients.
T_{comm}	ms	PCIe transfer time per decode step.
T_{plan}	ms	Predicted per-token decode latency.

activation transfer cost at the stage boundary (Table I). The total predicted per-token decode latency is:

$$T_{plan}(\pi_k) = T_{decode}^{CPU}(\{u_i\}_{i < k}) + T_{decode}^{GPU}(\{u_i\}_{i \geq k}) + T_{comm} \quad (9)$$

Each stage’s cost follows the roofline model: execution time is the maximum of compute time and memory-transfer time. For a stage on device $\mathcal{D}$, processing a batch of B sequences at context length c , the per-token decode time decomposes into a projection term and an attention term:

$$T_{decode} = T_{proj} + T_{attn} \quad (10)$$

(ii) Projection term: We read H , n_q , n_{kv} , d_h , d_{ff} , and n_ℓ (number of blocks in the stage) from the model config, and compute W_{param} analytically from the same config. P_{dec} and M

come from the microbenchmarks; $\eta_c, \eta_m \in (0, 1]$ are hardware efficiency coefficients we fit from profiling:

$$F_{proj} = 2B(2H(n_q d_h) + 2H(n_{kv} d_h) + 3H d_{ff}) \cdot n_\ell \quad (11)$$

where F_{proj} is the projection FLOP count for the stage. T_{proj} denotes the predicted per-token decode time spent in the linear projection part of a stage, i.e., the Q , K , V , and O projections together with the FFN linear layers across the n_ℓ transformer blocks assigned to that stage. Under the roofline model, this latency is the larger of the compute time and the memory-streaming time:

$$T_{proj} = \max\left(\frac{F_{proj}}{P_{dec} \eta_c}, \frac{W_{param}^{(stage)}}{M \eta_m}\right), \quad (12)$$

where $W_{param}^{(stage)}$ is the total parameter bytes streamed by that stage during one decode step.

(iii) Attention term: We read n_q , n_{kv} , d_h , and d_{sz} from the model config and take c from the workload description. We analytically derive the total floating-point work for the attention stage (F_{attn}), the KV-cache bytes (W_{kv}), and the predicted attention-term decode latency (T_{attn}) from these quantities. $\hat{M}$ is the effective KV bandwidth defined as:

$$\hat{M} = \alpha B_{L3} + (1 - \alpha) M, \quad \alpha = \min\left(1, \frac{C_{L3}}{W_{kv}}\right) \quad (13)$$

$$F_{attn} = 4B n_q c d_h \cdot n_\ell \quad (14)$$

$$W_{kv} = 2B n_\ell c n_{kv} d_h d_{sz} \quad (15)$$

$$T_{attn} = \max\left(\frac{F_{attn}}{P_{dec} \eta_c}, \frac{W_{kv}}{\hat{M} \eta_m}\right) \quad (16)$$

On the GPU, FlashAttention tiles Q , K , and V in SRAM, so no intermediate tensor is written to HBM; however, K and V are still read from HBM, so W_{kv} remains unchanged. On the CPU, the KV working set may fit in L3 cache at short contexts, making reads faster than DRAM. We blend DRAM and L3 bandwidth proportionally using α , the fraction of the KV working set that fits in cache.

As context grows, $\alpha \rightarrow 0$ and $\hat{M} \rightarrow M$ (pure DRAM bandwidth).

(iv) Communication term: We derive $W_{act} = B \cdot H \cdot d_{sz}$ analytically from the model config and take B_{link} for the host-to-device direction from the PCIe microbenchmark. For single-device plans, $T_{comm} = 0$; otherwise:

$$T_{comm} = \ell_{link} + \frac{W_{act}}{B_{link}} \quad (17)$$

ℓ_{link} is a constant per-crossing launch-and-sync overhead. It is the same on every mixed plan and does not change k . For Qwen3-8B, $W_{act} = 8$ KB, so the bandwidth term is $0.5 \mu s$ and T_{comm} is almost entirely ℓ_{link} , negligible relative to the compute terms.

Not every plan is feasible: the GPU-side weights plus KV cache (when KV offload is disabled) must fit in VRAM. We check this with:

$$W_{param}^{(gpu)}(k) + \mathbf{1}_{kv-GPU} \cdot W_{kv}^{total} \leq \text{VRAM}_{avail} \quad (18)$$

Algorithm 1 StrategySelector: Exhaustive Decode-Latency Partition Search

Require: Partition units $\{u_i\}_{i=0}^{U-1}$, device profiles $\mathcal{D}_{\text{cpu}}, \mathcal{D}_{\text{gpu}}$, link profile $\mathcal{L}$, VRAM_{avail}, workload (B, c) , $\mathbf{1}_{\text{kv-GPU}}$

Ensure: Optimal partition plan π^*

```

1:  $\pi^* \leftarrow \perp$ ;  $T^* \leftarrow \infty$ 
2: for  $k \leftarrow 0$  to  $U$  do
3:    $W_{\text{gpu}} \leftarrow \text{GPUFOOTPRINT}(k, \{u_i\}, B, c, \mathbf{1}_{\text{kv-GPU}})$ 
4:   if  $W_{\text{gpu}} > \text{VRAM}_{\text{avail}}$  then
5:     continue
6:   end if
7:    $T_{\text{cpu}} \leftarrow \text{DECODETIME}(\{u_i\}_{i < k}, \mathcal{D}_{\text{cpu}}, B, c)$ 
8:    $T_{\text{gpu}} \leftarrow \text{DECODETIME}(\{u_i\}_{i \geq k}, \mathcal{D}_{\text{gpu}}, B, c)$ 
9:   if  $k = 0$  or  $k = U$  then
10:     $T_{\text{comm}} \leftarrow 0$ 
11:   else
12:     $T_{\text{comm}} \leftarrow \mathcal{L}.\ell + W_{\text{act}}/\mathcal{L}.B$ 
13:   end if
14:    $T_{\text{plan}} \leftarrow T_{\text{cpu}} + T_{\text{gpu}} + T_{\text{comm}}$ 
15:   if  $T_{\text{plan}} < T^*$  then
16:      $T^* \leftarrow T_{\text{plan}}$ 
17:      $\pi^* \leftarrow (k, \mathcal{D}_{\text{cpu}}, \mathcal{D}_{\text{gpu}})$ 
18:   end if
19: end for
20: return  $\pi^*$ 

```

where $\mathbf{1}_{\text{kv-GPU}} = 0$ when KV offload is enabled (KV does not count against the GPU budget) and 1 otherwise. We evaluate T_{plan} for all feasible $k \in \{0, \dots, U\}$, where $U = L + 2$ is the number of partition units including the embedding and output projection. These candidates are the contiguous CPU-first unit splits: $k=0$ is all-GPU, $k=U$ is all-CPU, and each intermediate split carries a single per-token activation transfer. We return the plan that minimizes T_{plan} . Algorithm 1 summarizes this procedure.

The roofline prediction omits a context-independent per-block overhead (Python dispatch, kernel-launch latency, residual additions, normalization, and RoPE). We fold this into the search as a fixed per-block term $\bar{\delta}$, calibrated once per GPU from the all-GPU runs ($\bar{\delta}_{3090} \approx 0.79$ ms, $\bar{\delta}_{4060} \approx 0.14$ ms, $\bar{\delta}_{L40S} \approx 1.64$ ms), and rank splits by $\hat{T} = T_{\text{plan}} + N\bar{\delta}$, where N is the number of transformer blocks in the plan. Because $\bar{\delta}$ is added to every candidate, a constant offset between $\hat{T}$ and absolute latency shifts all plans together and does not move the argmin. On the heaviest configuration (Qwen3-32B, RTX 3090), predicted decode throughput rises monotonically toward the selected split `cpu(43u)+cuda(23u)`, which is both the cost-model argmin and the VRAM feasibility floor ($k < 43$ overflows the 24 GB card), so the chosen partition is the feasible split with the fewest CPU-side units. This coincidence holds for sequential CPU-then-GPU decode, so overflow k is the VRAM floor.

4) *Static Graph Reconstruction*: Once we know the optimal k , we reconstruct the FX graph as two independent per-device subgraphs, eliminating all placement logic from the token loop. We split the flat graph after the last node in block $k - 1$ and mark the activation tensor at that point as the output of the CPU subgraph and the input to the GPU subgraph. Each

subgraph is a standalone `torch.fx.GraphModule` that runs independently on its assigned device. Because placement is fixed at compile time, the token loop reduces to three operations: run the CPU subgraph, transfer the activation tensor across PCIe, and run the GPU subgraph. There are no per-token placement decisions, no weight movement, and no graph recompilation. Before the first request, we stream weights to their target devices one submodule at a time rather than loading the full model into CPU RAM at once, avoiding the transient memory spike that would otherwise exhaust host memory before any weights are sent to the GPU.

5) *CPU Execution and Roofline Validation*: The cost model predicts CPU decode time from the memory term W_{param}/M ; fp16 GEMV intensity is ≈ 1 FLOP/byte, and P_{dec} is set from the same M , so the compute branch does not change k . That prediction matches wall-clock time only if the CPU linear layers sustain about the profiled DRAM rate M .

On x86, PyTorch’s fp16 linear layer falls back to a scalar loop because AVX2 has no native FP16 fused multiply-accumulate instruction. On our Intel i7 CPU, this achieves only ≈ 3.7 GB/s, roughly one-twelfth of the 45 GB/s DRAM bandwidth and is well into the compute-bound regime. The roofline prediction would be wrong by the same factor.

HELM provides a JIT-compiled AVX2+F16C GEMV kernel that closes this gap. The kernel loads fp16 weights with F16C (`_mm256_cvtph_ps`), multiplies with AVX2 FMA (`_mm256_fmadd_ps`), and accumulates in fp32. On our i7, this achieves ≈ 50 GB/s, near the practical DDR4 bandwidth limit and $13\times$ faster than the PyTorch fallback. For prefill (sequence length > 1), a separate kernel casts fp16 weights to fp32 and calls MKL’s `cblas_sgemm`, achieving a $21\times$ speedup over PyTorch’s fp16 GEMM path. Both kernels are compiled on first use via `cpp_extension.load` and are automatically disabled on non-AVX2 hardware.

B. Runtime

With the partition plan fixed and the graphs compiled, the runtime maintains a simple invariant: every decode step follows the same three-stage sequence—CPU stage, PCIe transfer, GPU stage—the critical path that determines per-token latency, with no placement logic in the loop. These stages are strictly sequential because of a data dependency: the output of the CPU stage is the input to the GPU stage for the same token, so no within-step parallelism is possible. Maintaining this invariant still requires solving two problems that the compiler deferred. First, the KV cache grows by one entry per token per layer and will exhaust VRAM long before a practically useful context length is reached on commodity hardware. Second, once KV pages are evicted to CPU memory, computing attention over the full context requires reading pages that are no longer GPU-resident; without a contiguous KV buffer, standard attention kernels cannot be used directly. We address both problems in the following two subsections.

1) *KV Cache Management*: After the partition plan is finalized, the GPU holds a static set of transformer blocks, with their weights occupying most of the available VRAM, leaving only a small residual for KV entries. This residual is exhausted

quickly: every new token appends one KV entry per GPU-side block, so context length is effectively capped by how much VRAM remains after weights are placed. On commodity hardware, this limit is reached far below practically useful context lengths.

The obvious fix, a single large contiguous KV buffer, does not work here: it must be allocated in full at initialization, immediately consuming the residual VRAM and capping context at the same low limit. Growing it dynamically causes fragmentation that CUDA’s allocator does not handle efficiently, and evicting a contiguous buffer means evicting the entire KV history at once, discarding context irreversibly.

We manage KV memory in fixed-size *pages* instead, each holding `page_size` tokens of keys and values for a single layer (default 512). Pages can be independently allocated, evicted, and prefetched, giving fine-grained control over which parts of the context stay GPU-resident. A GPU page pool handles the active context window; a CPU-pinned (page-locked) memory pool serves as overflow. When GPU-resident KV exceeds the eviction threshold (default: 512 tokens), we evict the oldest full page to CPU memory with a `cudaMemcpyAsync`. We set that threshold so the current decode step can write its new KV entries without waiting for a free slot.

For Qwen3-8B on the RTX 4060, GPU-side KV is 4 KB per block per token. Across 16 GPU-resident blocks that is 64 KB per token, so a contiguous buffer exhausts the leftover VRAM in roughly 500 tokens. Paging keeps only the active pages GPU-resident and streams evicted pages from CPU RAM on demand.

2) *Streaming Paged Attention*: Paging solves the memory problem but creates an attention problem: at each decode step we need scores over the full context, yet some KV pages are no longer GPU-resident. Fetching every evicted page back into one contiguous KV tensor would undo paging. Skipping those pages would drop context tokens and produce incorrect output.

The root obstacle is the softmax denominator $\sum_j e^{s_j}$: it requires every score before any weight can be finalized, so attention cannot be split across pages naively. We resolve this using the FlashAttention online softmax identity [23], maintaining a running maximum score m , softmax denominator s , and weighted value sum o across pages, dividing only once after the final page. When a new page reveals a score larger than the current m , we rescale earlier contributions by $\exp(m_{\text{old}} - m_{\text{new}})$ before merging. This matches full-context attention. CPU-resident pages are copied to the GPU on a dedicated copy stream and consumed page by page; GPU-resident pages of that layer stay in place. The combined result is that context length scales with CPU RAM, not VRAM.

V. EVALUATION

We design our evaluation to answer five questions:

- 1) Can HELM run models that exceed GPU VRAM when all other backends fail?
- 2) Does HELM provide lower decode latency and higher decode throughput than feasible baselines on memory-constrained hardware?
- 3) How far does HELM extend the maximum supported output length?

- 4) Is exhaustive compile-time search cheap enough to run once per model–hardware pair?
- 5) Do HELM’s individual design choices—the AVX2 kernel and KV offload—each contribute, and to which metric?

A. Hardware Configurations

We evaluate on three systems chosen to cover the range of consumer and prosumer GPU memory budgets where CPU offloading is relevant:

- **4060**: RTX 4060 Laptop GPU (8 GB VRAM, 256 GB/s)¹ + Intel Core i7-13700H, 14 CPU cores, 16 GB DDR4 RAM.
- **3090**: NVIDIA RTX 3090 (24 GB VRAM, 936 GB/s) + AMD EPYC 7H12, 64 CPU cores / 128 threads, 125 GB DDR4 RAM. A mid-range server where smaller models fit in VRAM but larger ones do not.
- **L40S**: NVIDIA L40S (48 GB VRAM, 864 GB/s) + 24 vCPUs, 60 GB RAM. A GPU where most models fit entirely in VRAM.

B. Models

We test ten models across five architecture families. Qwen3 sets the size-scaling axis: its four dense SwiGLU + GQA variants, Qwen3-4B, 8B, 14B, and 32B (8, 16, 28, and 64 GB of fp16 weights) [24], range from fitting entirely in VRAM to overflowing every test system, covering both HELM’s target regime and the baseline-equivalent case. Because the partition plan depends on architectural parameters (hidden size, FFN expansion ratio, GQA group count, vocabulary size), the other four families vary exactly those parameters: LLaMA-2-13B (multi-head attention, 32K vocabulary) [25] and LLaMA-3.1-8B (GQA, 128K vocabulary) [26]; Mistral-Nemo-12B (GQA, 22.8 GB of fp16 weights) [27], OLMo-2-13B (QK-Norm) [28], and Gemma-2-9B/27B (sliding-window attention) [29].

C. Baselines

We compare HELM against three backends that represent the current SOTA for CPU offloading and memory-constrained inference:

- **vLLM** [6] v0.17.1, PagedAttention, fp16, default GPU memory utilization (0.90).
- **Accelerate** [8] `device_map='auto'`, fp16.
- **DeepSpeed** [9] ZeRO-Inference, fp16, CPU offload enabled.

All baselines run with default settings. SGLang [7] is omitted from the quantitative comparison: on overflow it does not add a feasible backend beyond default vLLM, which fails at weight load (OOM) on most overflow cells and at initialization (×) on the tabulated RTX 4060 models. We run HELM with automatic partitioning, KV offload, the AVX2 kernel, and fp16

¹Bandwidths listed here are manufacturer specifications. HELM’s cost model uses profiled values measured at runtime startup; for example, the RTX 4060 achieved a sustained 218 GB/s in our microbenchmark (see §IV-A2).

TABLE II
DECODE THROUGHPUT (TOK/S) / TTFT P50 (S), OUTPUT LENGTH 128, BATCH=1. BOLD = BEST PER ROW. SEE NOTES FOR SYMBOL KEY.

GPU	Model	HELM tok/s / TTFT	Accel tok/s / TTFT	DeepSpeed tok/s / TTFT	vLLM tok/s / TTFT	HELM vs. best base. × tok/s
RTX 4060 (8 GB)	Qwen3-4B [†]	18.8/0.340	5.7/0.201	×	×	3.3×
	Qwen3-8B [†]	3.8/6.3	0.6/1.6	×	×	6.3×
RTX 3090 (24 GB)	Qwen3-4B	87.1/0.019	25.7/0.041	26.7/0.040	87.0/0.018	1.0×
	Qwen3-8B	50.3/0.028	25.8/0.042	26.5/0.040	50.3/0.027	1.0×
	Qwen3-14B [†]	8.1/4.7	1.3/0.782	×	OOM	6.1×
	Qwen3-32B [†]	1.8/11.7	0.2/5.3	×	OOM	9.1×
	LLaMA-2-13B [†]	10.6/0.762	2.3/0.457	OOM	OOM	4.6×
	LLaMA-3.1-8B	51.8/0.027	44.8/0.023	44.6/0.024	51.8/0.027	1.0×
	Mistral-Nemo-12B	30.5/0.035	2.5/0.405	29.5/0.037	KV	1.0×
	OLMo-2-13B [†]	9.1/2.7	1.4/0.807	OOM	OOM	6.6×
	Gemma-2-9B	40.6/0.033	21.0/0.050	21.1/0.050	40.7/0.033	1.0×
	Gemma-2-27B [†]	1.2/10.9	0.3/3.8	OOM	OOM	4.8×
L40S (48 GB)	Qwen3-4B	83.6/0.016	50.6/0.022	51.5/0.021	83.6/0.015	1.0×
	Qwen3-8B	46.7/0.026	38.1/0.030	38.2/0.029	46.7/0.026	1.0×
	Qwen3-14B	26.0/0.045	23.0/0.050	22.9/0.050	26.0/0.045	1.0×
	Qwen3-32B [†]	4.8/6.5	0.5/2.2	×	OOM	10.4×
	LLaMA-2-13B	28.5/0.042	25.6/0.045	25.6/0.045	28.5/0.042	1.0×
	LLaMA-3.1-8B	47.6/0.026	40.8/0.028	41.3/0.027	47.6/0.026	1.0×
	Mistral-Nemo-12B	31.2/0.038	28.0/0.041	28.0/0.041	31.2/0.038	1.0×
	OLMo-2-13B	27.7/0.042	23.8/0.048	23.9/0.048	27.7/0.042	1.0×
	Gemma-2-9B	38.4/0.032	31.0/0.036	31.1/0.036	38.5/0.032	1.0×
	Gemma-2-27B [†]	5.1/0.220	0.6/1.7	OOM	OOM	8.3×

OOM: weight-load out-of-memory. ×: initialization or compile/profile failure. **KV**: KV-allocation OOM (vLLM, Mistral-Nemo-12B, RTX 3090). [†]: overflow (fp16 weights exceed VRAM); these nine rows enter the 6.2× geo-mean. When weights fit VRAM, GPU layers run in vLLM with HELM’s paged KV manager. Overflow models (>VRAM) run the compiled CPU+GPU pipeline. Last column: HELM speedup over the best feasible baseline, computed from unrounded measurements; it can differ from the ratio of the rounded cells, most visibly where a baseline rounds to one decimal (e.g., 1.8/0.2 prints as 9.1×, not 9.0×). TTFT in seconds. All 3090 rows are measured on the host described in §V-A.

precision enabled across all experiments, except the Gemma-2 family, which we run in bf16 on every backend because Gemma-2 is numerically unsafe in fp16. When fp16 weights fit in VRAM, GPU-resident layers execute in vLLM with HELM’s paged KV-cache manager. When weights exceed VRAM, HELM runs the compiled CPU+GPU pipeline.

D. Metrics and Methodology

Unless otherwise noted, experiments use a fixed input prompt of 128 tokens drawn from a generic instruction-following template. We report the median (p50) over 10 independent requests, since the first request includes one-time overhead (KV allocator initialization, CUDA graph warm-up) that inflates the mean.

Our primary metric is *decode throughput* (output tokens per second, single request, batch=1), because decode is the bottleneck for interactive use on commodity hardware. At batch size 1 and fixed output length, per-token latency and throughput are inversely related, so we treat them as equivalent throughout. We also report time-to-first-token (TTFT) and maximum supported output length. Ablation studies isolate the contribution of each design component across both overflow configurations (weights do not fit in VRAM) and in-VRAM configurations.

VI. EXPERIMENTAL CLAIMS

A. HELM is 6.2× geomean (up to 10.4×) faster in decode tok/s than the strongest feasible baseline when weights exceed VRAM

Table II compares decode throughput (tok/s) and TTFT for every model–hardware pair that fits in combined CPU+GPU

memory. The full 10-model suite appears on the RTX 3090 and L40S; on the RTX 4060 (8 GB VRAM, 16 GB host RAM, 24 GB combined) only Qwen3-4B and Qwen3-8B leave enough headroom for fp16 weights plus runtime state, so those are the only 4060 rows. The other eight 4060 pairs are absent because they do not fit that budget: five exceed 24 GB in weights alone (Qwen3-14B, Qwen3-32B, LLaMA-2-13B, OLMo-2-13B, and Gemma-2-27B), while LLaMA-3.1-8B (16 GB), Gemma-2-9B (18 GB), and Mistral-Nemo-12B (22.8 GB) leave insufficient host DRAM for KV cache and allocator overhead. **OOM** indicates failure at load time, × indicates an initialization or compile/profile failure, and **KV** indicates KV-allocation OOM after a successful weight load; a backend so marked cannot be used for that configuration at output length 128. Default vLLM fails at *weight load* on the overflow cells marked OOM, at initialization (×) on both tabulated RTX 4060 models, and at *KV allocation* on Mistral-Nemo-12B on the RTX 3090; Accelerate fails when fp16 weights exceed combined CPU+GPU memory; DeepSpeed fails at weight load on some models and at *engine initialization* on others.

On the RTX 4060, Accelerate and HELM load both tabulated models (Qwen3-4B and Qwen3-8B) via CPU+GPU partitioning; default vLLM and DeepSpeed fail to initialize on both. On the RTX 3090, HELM and Accelerate run all ten models. Default vLLM runs the four that fit in VRAM with residual KV headroom (Qwen3-4B, Qwen3-8B, LLaMA-3.1-8B, Gemma-2-9B) and fails at weight load on Qwen3-14B, Qwen3-32B, LLaMA-2-13B, OLMo-2-13B, and Gemma-2-27B; on Mistral-Nemo-12B the 22.8 GB weights load and leave 27 MB free, then vLLM fails at KV allocation.

TABLE III

DECODE THROUGHPUT VS. OUTPUT LENGTH S FOR MISTRAL-NEMO-12B ON THE RTX 3090 AT `cpu(1u)+cuda(41u)`, $in=128$, $BATCH=1$. THE $S=128$ CELL IS THE TABLE II MEASUREMENT. THE COST MODEL DOES NOT USE S TO PICK THE SPLIT.

Output length S	tok/s
128	30.5
512	19.2
4096	7.2

DeepSpeed runs those four in-VRAM models plus Mistral-Nemo-12B, fails to initialize on Qwen3-14B and Qwen3-32B, and OOMs at weight load on LLaMA-2-13B, OLMo-2-13B, and Gemma-2-27B. On the five overflow models where Accelerate is the only other feasible backend (Qwen3-14B, Qwen3-32B, LLaMA-2-13B, OLMo-2-13B, Gemma-2-27B), HELM is 4.6–9.1 $\times$ faster than Accelerate. On Mistral-Nemo-12B the plan keeps every transformer block on the GPU (`cpu(1u)+cuda(41u)`, embedding only on CPU); HELM reaches 30.5 tok/s versus DeepSpeed’s 29.5 tok/s.

On the L40S, HELM and Accelerate run all ten models. Default vLLM runs the eight that fit in VRAM (Qwen3-4B, 8B, and 14B; LLaMA-2-13B; LLaMA-3.1-8B; Mistral-Nemo-12B; OLMo-2-13B; Gemma-2-9B) and OOMs at weight load on Qwen3-32B and Gemma-2-27B. DeepSpeed matches that eight-model set, fails to initialize on Qwen3-32B, and OOMs on Gemma-2-27B. On those two overflow models, HELM reaches 4.8 and 5.1 tok/s versus Accelerate’s 0.5 and 0.6 tok/s (10.4 $\times$ and 8.3 $\times$); Qwen3-32B uses a `cpu(18u)+cuda(48u)` partition with a 6.5s TTFT, the largest gain we observe. On every overflow configuration that any backend loads, HELM achieves the highest decode throughput. Accelerate also loads these models, but HELM is 3.3–10.4 $\times$ faster. On 32B-class overflow models, HELM’s 1.8–4.8 tok/s is better suited to batch or background generation than to interactive use.

Averaged across the nine overflow configurations marked $\dagger$ in Table II, HELM delivers a **6.2 $\times$** geometric-mean throughput improvement over the strongest feasible baseline (speedup column). The throughput gap comes from Accelerate’s per-hook dispatch, which introduces Python-level synchronization at every layer boundary and streams weights on each forward pass; HELM eliminates both by using static subgraphs with weights placed once.

Table II reports decode at output length $S=128$. A generation-length sweep on Mistral-Nemo-12B (RTX 3090, the same `cpu(1u)+cuda(41u)` plan) falls from 30.5 tok/s at $S=128$ to 19.2 tok/s at 512 and 7.2 tok/s at 4096 (Table III). $S=128$ remains the setting used to select the split: the cost model ranks plans by per-token decode latency with prefill excluded (§IV-A3), so the chosen split does not depend on S . The sweep characterizes throughput *at* that split as the KV cache grows; it is a different axis from Fig. 6, which varies prompt context under paging.

HELM’s TTFT is higher when the CPU stage is large (11.7s vs. Accelerate’s 5.3s on RTX 3090 Qwen3-32B at $in=out=128$) because prefill runs in place on CPU-resident

layers; decode is 9.1 $\times$ faster, so the extra prefill is repaid within two output tokens. Fig. 4 sweeps prompt length at the auto-selected `cpu(43u)+cuda(23u)` split on that same 3090 host. TTFT is not flat: p50 is 11.7, 14.7, 22.2, 51.2, 105.2, and 243.1s at 128, 256, 512, 1024, 2048, and 4096 input tokens, a 20.8 $\times$ rise over the 32 $\times$ input range. From 512 tokens upward, each doubling of the prompt multiplies TTFT by 2.1–2.3 $\times$. The projection-to-attention FLOP crossover for Qwen3-32B lies in the tens of thousands of tokens, well beyond this range, so projections still dominate and the scaling stays near-linear in prompt length. The cost model selects the split on per-token decode latency, with prefill excluded (§IV-A3), so the sweep changes only the TTFT at that split, not which split is chosen. Optimizing prefill-dominated workloads such as RAG and summarization is future work.

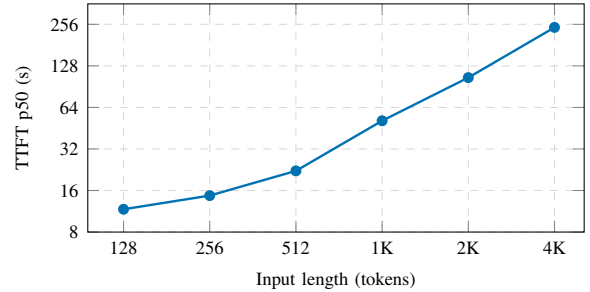


Fig. 4. TTFT p50 vs. input length for Qwen3-32B on the RTX 3090 at the auto-selected `cpu(43u)+cuda(23u)` split (same host as Table II). The 128-token point is the 11.7s table cell. TTFT rises 20.8 $\times$ over a 32 $\times$ input range; from 512 tokens upward each doubling multiplies TTFT by 2.1–2.3 $\times$.

B. HELM extends maximum output length by 5.9 $\times$ geometric (up to 256 $\times$) on the Qwen3 family

Fig. 5 shows, across all three systems, the longest output sequence each backend can produce. The y-axis is maximum output tokens on a log₂ scale; each bar group is an LLM, and each bar is a backend. Floor bars mark the first failing cause: W (weight-load OOM), I (initialization or compile/profile failure), A (KV-cache allocation OOM), or F (combined CPU+GPU memory exceeded).

In panel (a) (RTX 4060), DeepSpeed and vLLM produce no output at all; their bars sit at the axis floor (I) on all four models. Accelerate reaches 1K tokens on Qwen3-4B and only 128 tokens on Qwen3-8B before exhausting VRAM. HELM reaches 8K and 4K on the 4B and 8B models, respectively, an **8 $\times$** extension on Qwen3-4B and a **32 $\times$** extension on Qwen3-8B. In panel (b) (RTX 3090), HELM sustains 32K tokens on Qwen3-14B, versus Accelerate’s 128, a **256 $\times$** advantage; for Qwen3-32B, HELM reaches 4096 tokens; DeepSpeed fails to initialize, vLLM runs out of memory at weight load, and Accelerate exhausts memory at KV-cache allocation (A). In panel (c) (L40S), for models up to 14B that fit in VRAM, HELM matches vLLM (both reach 32K on 4B, 8B, and 14B); for Qwen3-32B, HELM reaches 8K, Accelerate generates 1K tokens, DeepSpeed fails to initialize, and vLLM fails to load. Averaged over the nine Qwen3 pairs in Fig. 5 that have a HELM bar and at least one baseline bar, HELM extends maximum output length by **5.9 $\times$** over the strongest feasible baseline.

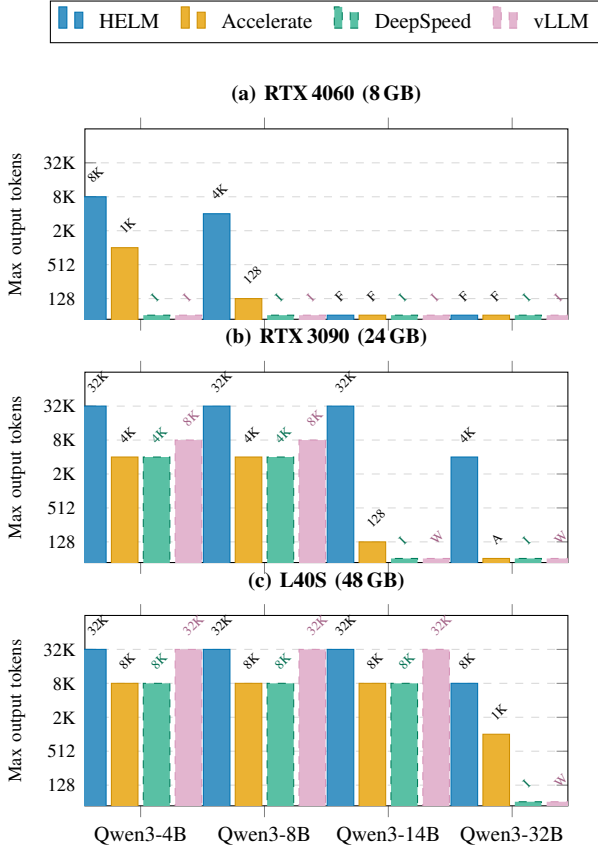


Fig. 5. Maximum supported output length per backend (log₂ scale). Floor bars mark the first failing cause: W, weight-load OOM; I, initialization or compile/profile failure; A, KV-cache allocation OOM; F, combined CPU+GPU memory exceeded. (a) RTX 4060 (8 GB): DeepSpeed and vLLM fail to initialize on both models that fit; Qwen3-14B/32B exceed combined CPU+GPU memory for HELM and Accelerate (F), while DeepSpeed and vLLM fail earlier at initialization (I). HELM extends maximum output 8 $\times$ beyond Accelerate on Qwen3-4B (8K vs. 1K) and 32 $\times$ on Qwen3-8B (4K vs. 128). (b) RTX 3090 (24 GB): HELM is the only backend that sustains long-output generation on 32B (4K tokens); Accelerate’s eager KV allocation runs out of memory (A); vLLM reaches 8K on 4B and 8B and fails at weight load (W) on 14B and 32B; DeepSpeed fails to initialize (I) on 14B and 32B. (c) L40S (48 GB): HELM matches vLLM on 4B–14B and extends to 8K for 32B via CPU+GPU partitioning; on 32B, Accelerate reaches 1K, DeepSpeed fails to initialize, and default vLLM fails at weight load.

C. HELM’s exhaustive partition search is a one-time cost under 0.01 s; end-to-end compile stays under 60 s

On Qwen3-32B (RTX 3090), the five compiler stages take 15.7 s: device profiling 13.2 s, graph analysis 1.8 s, partition search and graph construction under 0.01 s each, and warm kernel JIT 0.73 s. The exhaustive search over contiguous splits is therefore negligible; profiling dominates and is cached across models on the same machine. Across all evaluated configurations, end-to-end compilation completes in under 60 seconds. The AVX2 JIT is a one-time `cpp_extension` build that could ship as a precompiled binary.

VII. ABLATION AND PERTURBATION STUDIES

Each of HELM’s design choices interacts with the cost model’s partition plan. An all-GPU configuration (e.g., Qwen3-4B/8B on the RTX 3090) behaves very differently from one with a large CPU stage (e.g., Qwen3-4B on the RTX 4060, or Qwen3-14B on the RTX 3090). We therefore

TABLE IV
HELM VS. LLAMA.CPP ON LLAMA-2-13B, RTX 3090, IN=OUT=128, DECODE TOK/S (LLAMA.CPP P50; HELM MEAN, $n=20$; THE TWO STATISTICS AGREE TO WITHIN 1% SINCE DECODE VARIATION IS BELOW 1%). NGL IS LLAMA.CPP’S MANUAL COUNT OF GPU-RESIDENT LAYERS.

Config	Precision	tok/s
llama.cpp ngl=-1 (all GPU)	Q4_K_M	74.5
llama.cpp ngl=0 (all CPU)	Q4_K_M	7.45
llama.cpp ngl=20	Q4_K_M	13.1
llama.cpp ngl=20	fp16	4.29
llama.cpp ngl=30	fp16	7.16
llama.cpp ngl=34 (best fp16)	fp16	9.78
HELM (auto)	fp16	10.6

report a direct baseline comparison and a set of ablations across both platforms and all available models.

A. Comparison with llama.cpp and quantized inference

llama.cpp supports static CPU–GPU layer placement through its manual `--n-gpu-layers` (ngl) parameter, and quantized inference dominates local LLM deployment. We therefore compare both on LLaMA-2-13B on the RTX 3090 (Table IV).

In fp16, HELM with automatic partitioning reaches 10.6 tok/s and beats every ngl setting we swept. HELM’s plan (cpu(6)+cuda(36) units) places the embedding and the first five of the 40 transformer layers on the CPU and keeps the remaining 35 layers plus the output projection on the GPU. The best hand-tuned fp16 setting reaches 9.78 tok/s at ngl=34; the manual settings span a 2.3 $\times$ range that the user must tune per model and card.

The 4-bit Q4_K_M path reaches 74.5 tok/s by fitting a 4-bit copy of the model entirely in VRAM, and even a partially offloaded Q4 setting (ngl=20, 13.1 tok/s) outpaces fp16 HELM. Quantization shrinks the bytes streamed on every weight access, an advantage independent of placement. That is a different precision and accuracy target, which HELM does not yet exploit; extending the cost model and the AVX2 kernel to quantized weights is future work.

Relative to llama.cpp, HELM contributes two capabilities. First, automatic partition selection from a hardware-calibrated roofline model: llama.cpp’s ngl is a manual per-model, per-card parameter, and the 2.3 $\times$ spread above is the cost of getting it wrong. Second, demand-paged KV offload with streaming attention integrated into the compiled graph, which extends context beyond VRAM independently of weight placement.

B. KV Offload

The benefit of KV offload depends on whether the partition is all-GPU or split across CPU and GPU. When all layers fit on the GPU, disabling KV offload cuts throughput sharply, by 51% and 52% for Qwen3-4B and 8B on the RTX 3090 and by 79%, 78%, and 75% for the 4B, 8B, and 14B models on the L40S, because the runtime then falls back from HELM’s preallocated paged-KV buffers to the HuggingFace

`DynamicCache`, which re-concatenates each layer’s key and value tensors at every decode step. In this all-GPU regime the KV cache never pages, so no PCIe transfer exists to be eliminated; the measured gap is the cost of the fallback’s per-step cache reallocation, not a benefit of the paging mechanism itself. In mixed CPU–GPU partitions, the gap shrinks and its sign tracks how much of the decode step runs GPU-side, where the preallocated buffer replaces the fallback’s per-step reallocation. On Qwen3-32B on the RTX 3090, KV offload still improves throughput by 36%, because the GPU stage is a substantial share of decode latency. Where the CPU stage instead dominates the decode step, GPU-side handling is off the critical path, so the change is small and can reverse: disabling KV offload increases throughput by only 9.7% (Qwen3-14B, 3090), 4.0% (Qwen3-8B, 4060), and 0.8% (Qwen3-4B, 4060). In those CPU-bound cases, KV offload’s primary value is extending context length rather than throughput.

C. Attention kernel (paged SDPA vs. contiguous KV)

We compare paged-GPU attention against a single contiguous-KV buffer in the all-GPU regime, the only regime where a contiguous baseline is feasible. Where both paths dispatch to `FlashAttention-2` through `scaled_dot_product_attention`, the gap stays within 2.3%. On the RTX 3090, the paged path reaches 39.69 tok/s versus 40.56 tok/s for the contiguous buffer on LLaMA-3.1-8B, and 35.69 versus 36.50 on Qwen3-8B, gaps of 2.2% and 2.3%, respectively. The paged data structure therefore adds at most a small overhead, within measurement noise.

D. CPU Kernel (AVX2 vs. PyTorch Fallback)

The kernel’s impact scales with the number of CPU-assigned layers. When all layers reside on the GPU, it has no measurable effect (at or below 0.4% on the RTX 3090 4B/8B and on the L40S up to 14B). When a significant CPU stage is present, the scalar PyTorch fp16 fallback dominates: disabling the kernel drops throughput by 39.6% on the RTX 4060 Qwen3-4B (`cpu(4u)`), 87.6% on the RTX 4060 Qwen3-8B (`cpu(22u)`), 58.8% on the L40S Qwen3-32B (`cpu(18u)`), and 87% on the RTX 3090 Qwen3-32B (`cpu(43u)`). This matches the roofline analysis of §IV-A5: without the kernel, PyTorch’s fp16 path reaches only ≈ 3.7 GB/s on our i7, roughly one-twelfth of the 45 GB/s DRAM ceiling, breaking the memory-bandwidth-bound assumption that the cost model relies on for any large CPU stage.

E. Partition topology (interleaved and GPU-first placements)

We measure the placement alternatives excluded from the search space (§IV-A3) on the RTX 3090 (`in=out=128`, $n=5$). We hold the layer counts fixed, so per-device compute and weight footprint are identical, and vary only the placement, that is, how many times the activation crosses the CPU-to-GPU boundary per token. We run two overflow regimes, where the memory budget already forces CPU offload. LLaMA-2-13B has 26 GB of fp16 weights, which exceed the 24 GB card;

for this ablation we fix a partition that places 8 transformer layers and the embedding on the CPU (peak GPU 19.9 GB), slightly heavier than the auto-selected `cpu(6u)` plan, so the interleaved variants have layers to redistribute. Qwen3-32B offloads 42 of its 64 layers to the CPU (peak GPU 22.0 GB), the regime HELM targets. Interleaving the same layers does not help: splitting the CPU layers into 2 and 4 blocks raises boundary crossings per token from 1 to 3 to 7, yet decode throughput on the 13B falls only slightly and monotonically, from 10.17 to 10.02 to 9.97 tok/s (1.9%), with TTFT rising by at most 2.7%. Each crossing costs about 0.33 ms (the per-token decode step lengthens from 98.3 to 100.3 ms); this is ℓ_{link} in Eq. 17. At batch 1 the crossing tensor is one token’s hidden state (10 KB at $H=5120$) and the move is latency-bound rather than bandwidth-bound: those bytes are about $0.3 \mu\text{s}$ of PCIe Gen4 transfer, so the cost is launch-and-sync overhead, paid serially because HELM runs the stages sequentially without overlap. On the 32B, that fixed overhead is negligible relative to a much longer step: the per-token decode is about 560 ms, so the same 1–7-crossing sweep changes decode throughput only within run-to-run noise (about 3%, non-monotonic). Adding crossings never helps in either regime.

The ordering can also matter, but in our tested topology ablation, GPU-first is never faster. By GPU-first, we mean the reverse of HELM’s CPU-then-GPU ordering: the GPU runs the leading transformer blocks and the activation then crosses back to the CPU, which holds the trailing blocks and the vocabulary output projection (stage plan `cpu(1u) → cuda(22u) → cpu(43u)`, so the output unit, not the prefix, lands on the slow device). Running that projection on the CPU reduces throughput by 3.9% on the 152K-vocabulary Qwen3-32B and 1.8% on the 32K-vocabulary LLaMA-2-13B (9.99 vs. 10.17 tok/s), the gap tracking the size of that output GEMM, the largest decode matmul. CPU-then-GPU is never slower in this ablation, so we fix that ordering. Sub-layer splits cannot raise arithmetic intensity, since the decode GEMMs are already bandwidth-bound, and interleaving is an empirical upper bound on the benefit of finer-grained splits. The contiguous CPU-then-GPU family with a single crossing is therefore the right search space, including in the overflow regime that HELM targets. §IV-A3 reports the selected plan as the argmin over that family.

F. Throughput with paging active

On Mistral-Nemo-12B (RTX 3090), whose 22.8 GB of fp16 weights nearly fill the card, the KV cache stays GPU-resident up to 4K context (5.7 tok/s measured at 4K, with nothing evicted). Past the GPU KV budget, paging activates and decode throughput falls as host page traffic grows, to 1.46 tok/s at 8K and 0.37 tok/s at 16K (Fig. 6). At 16K the run evicts 8.1 GB and prefetches 584 GB of KV pages cumulatively, while peak GPU memory stays bounded, rising only from 22.3 to 23.4 GB as the KV cache grows $4\times$. vLLM loads this model but cannot run it at 16K: the weights leave 27 MB of VRAM free and it fails at KV allocation. A controlled run on LLaMA-2-13B (GPU KV watermark 1,536 MiB; 9.6 tok/s resident

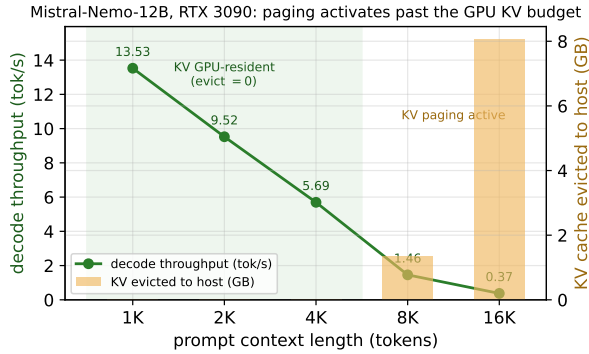


Fig. 6. Decode throughput vs. context length for Mistral-Nemo-12B on the RTX 3090, with KV paging active past the GPU KV budget. Throughput falls as host page traffic grows, while peak GPU memory stays bounded near 23.4GB; vLLM fails at KV allocation at 16K.

peak) isolates the dominant per-token cost. At paging onset (context 2,560), paged-KV PCIe traffic is already 225 MB per decode step—the 35 GPU-stage layers hold 1,750 MiB of evictable KV at that context, and the 1,536 MiB watermark leaves 214 MiB to restream every step—versus the fixed 10 KB hidden-state transfer, which crosses once. Decode throughput falls below half its 9.6 tok/s resident peak by context 3,072.

Paging is correctness-preserving: under greedy decoding, the output is token-identical with and without offload over 2048 generated tokens (2048 of 2048 tokens match), confirming the online-softmax reconstruction (§IV-B2) is exact, and a needle-in-a-haystack probe confirms that the planted span is recalled correctly under multi-gigabyte eviction at context lengths up to 65K.

G. Page size, with paging active

This sweep uses GPUs outside the main evaluation set (Quadro RTX 5000 16GB, RTX 2080 Ti 11 GB, RTX A6000 48GB) where a paging regime can be forced cleanly. When weights or the KV cache exceed VRAM, KV pages spill to host memory and only KV traffic crosses PCIe. Page size matters, and decode throughput rises monotonically with page size (Table V). The evicted volume is essentially constant across page sizes; larger pages move the same data in fewer, larger transfers and amortize the per-call overhead, with diminishing returns toward a transfer-bound plateau (the last doubling adds only 4% on the RTX A6000). There is no interior optimum: within the paging regime, larger pages are strictly better, and the next doubling to 1024 either exhausts VRAM on the memory-constrained cards or grows the page enough that the cache no longer pages. We therefore default to 512, not as a tuned optimum, but as the largest page that fits the memory-constrained budget.

VIII. CONCLUSION

We presented HELM, a compiler and runtime for heterogeneous LLM inference on memory-constrained CPU+GPU systems. By recasting CPU–GPU inference as a compile-time optimization problem and searching exhaustively over all feasible contiguous splits, HELM finds the latency-optimal

TABLE V

DECODE THROUGHPUT (TOK/S) VS. KV PAGE SIZE WITH PAGING ACTIVE; CPU-RESIDENT LAYER COUNT IN PARENTHESES. THROUGHPUT RISES WITH PAGE SIZE BECAUSE THE EVICTION AND PREFETCH CALL COUNT FALLS WHILE THE EVICTED BYTES STAY ROUGHLY CONSTANT. PAGE 1024 EXHAUSTS VRAM (OOM) ON THE MEMORY-CONSTRAINED CARDS; THE RTX 2080 Ti P256 CELL WAS NOT RUN (N/A), SO THAT SWEEP IS INCOMPLETE. ALL ROWS $n=3$.

GPU / model (CPU layers)	p64	p128	p256	p512	p1024
RTX 5000 / L3.1-8B (1)	0.74	1.29	2.08	3.16	n/a
RTX 3090 / L2-13B (6)	2.86	3.03	3.66	3.91	OOM
RTX 2080Ti / L3.1-8B (13)	1.10	1.19	n/a	1.35	OOM
RTX A6000 / Qwen3-32B (18)	0.39	0.46	0.51	0.53	n/a

partition in the contiguous-split family, subject to the VRAM-feasibility constraint. In our evaluation across three GPUs and ten models from five architecture families (4B–32B parameters), HELM runs every memory-constrained configuration where at least one baseline is feasible and achieves the highest decode throughput on each overflow configuration. Where a baseline is feasible, HELM achieves up to **10.4×** higher decode throughput (equivalently, lower per-token latency), with a geometric mean of **6.2×** over the strongest feasible baseline across the nine overflow configurations, and extends maximum output length by up to **256×** (geometric mean **5.9×** on the nine feasible Qwen3 pairs in Fig. 5). A principled compile-time approach, grounded in hardware measurement, sustains efficient inference on consumer hardware without model quantization, multi-GPU infrastructure, or cloud resources, and scales automatically as DRAM and interconnect bandwidth improve, since HELM’s profiler measures the hardware at startup. Three directions remain open. An overlapped, pipelined executor that runs the CPU and GPU stages concurrently across micro-batches or chunked prefill could hide one stage behind the other—the one setting in which interleaved placements could become profitable (§VII-E). Dynamic re-profiling would extend the one-shot hardware profile to settings with co-located contention and to multi-hour serving (§IV-A2). Finally, Mixture-of-Experts models, whose sparse per-token expert activation interacts differently with layer-level partitioning, are outside the scope of this work.

ACKNOWLEDGMENT

The authors used OpenAI ChatGPT to assist with language editing and refinement of author-written text. Anthropic Claude Code and OpenAI Codex were used to assist with debugging, code development, and implementation tasks related to the experimental code. The design and implementation of the compiler and runtime system, the experimental methodology, interpretation of the results, and the conclusions are the authors’ own work. All AI-assisted output was reviewed, tested, and verified by the authors, who take full responsibility for the content of this article.

REFERENCES

- [1] “Lm studio.” [Online]. Available: <https://lmstudio.ai/>
- [2] “ollama.” [Online]. Available: <https://ollama.com/>

- [3] Openclaw, "Openclaw/openclaw: Your own personal ai assistant. any os. any platform. the lobster way." [Online]. Available: <https://github.com/openclaw/openclaw>
- [4] Z. Lu, X. Li, D. Cai, R. Yi, F. Liu, W. Liu, J. Luan, X. Zhang, N. D. Lane, and M. Xu, "Demystifying small language models for edge deployment," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 14 747–14 764. [Online]. Available: <https://aclanthology.org/2025.acl-long.718/>
- [5] B. Kubwimana and Q. Huang, "Edgereasoning: Characterizing reasoning llm deployment on edge gpus," 2025. [Online]. Available: <https://arxiv.org/abs/2511.01866>
- [6] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th symposium on operating systems principles*, 2023, pp. 611–626.
- [7] L. Zheng, L. Yin, Z. Xie, C. Sun, J. Huang, C. H. Yu, S. Cao, C. Kozyrakis, I. Stoica, J. E. Gonzalez *et al.*, "Sglang: Efficient execution of structured language model programs," *Advances in neural information processing systems*, vol. 37, pp. 62 557–62 583, 2024.
- [8] S. Gugger, L. Debut, T. Wolf, P. Schmid, Z. Mueller, S. Mangrulkar, M. Sun, and B. Bossan, "Accelerate: Training and inference at scale made simple, efficient and adaptable," *Computer software*. *Hugging Face*. <https://github.com/huggingface/accelerate>, 2022.
- [9] R. Y. Aminabadi, S. Rajbhandari, A. A. Awan, C. Li, D. Li, E. Zheng, O. Ruwase, S. Smith, M. Zhang, J. Rasley *et al.*, "DeepSpeed-Inference: enabling efficient inference of transformer models at unprecedented scale," in *SC22: International Conference for High Performance Computing, Networking, Storage and Analysis*. IEEE, 2022, pp. 1–15.
- [10] Y. Sheng, L. Zheng, B. Yuan, Z. Li, M. Ryabinin, B. Chen, P. Liang, C. Ré, I. Stoica, and C. Zhang, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proceedings of the International Conference on Machine Learning (ICML)*, 2023, pp. 31 094–31 116.
- [11] G. Gerganov, "llama.cpp," <https://github.com/ggerganov/llama.cpp>, 2023.
- [12] J. He and J. Zhai, "Fastdecode: High-throughput gpu-efficient llm serving using heterogeneous pipelines," *arXiv preprint arXiv:2403.11421*, 2024.
- [13] X. Jiang, Y. Zhou, S. Cao, I. Stoica, and M. Yu, "Neo: Saving gpu memory crisis with cpu offloading for online llm inference," *Proceedings of Machine Learning and Systems*, vol. 7, 2025.
- [14] Y. Xiong, H. Wu, C. Shao, Z. Wang, R. Zhang, Y. Guo, J. Zhao, K. Zhang, and Z. Pan, "Layerkv: Optimizing large language model serving with layer-wise kv cache management," *arXiv preprint arXiv:2410.00428*, 2024.
- [15] C. Luo, Z. Cai, H. Sun, J. Xiao, B. Yuan, W. Xiao, J. Hu, J. Zhao, B. Chen, and A. Anandkumar, "Headinfer: Memory-efficient llm inference by head-wise offloading," *arXiv preprint arXiv:2502.12574*, 2025.
- [16] Y. Liu, Y. Cheng, J. Yao, Y. An, X. Chen, S. Feng, Y. Huang, S. Shen, R. Zhang, K. Du *et al.*, "Lmcache: An efficient kv cache layer for enterprise-scale llm inference," *arXiv preprint arXiv:2510.09665*, 2025.
- [17] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "Hetegen: Efficient heterogeneous parallel inference for large language models on resource-constrained devices," *Proceedings of Machine Learning and Systems*, vol. 6, pp. 162–172, 2024.
- [18] C. Yu, T. Wang, Z. Shao, L. Zhu, X. Zhou, and S. Jiang, "Twinpilots: A new computing paradigm for gpu-cpu parallel llm inference," in *Proceedings of the 17th ACM international systems and storage conference*, 2024, pp. 91–103.
- [19] A. Ukarande, D. Shekhar, M. Blackstein, and R. Rangan, "Efficient, VRAM-constrained xLM inference on clients," in *Proceedings of the 9th Conference on Machine Learning and Systems (MLSys)*, 2026.
- [20] Y. Hu, C. Zhang, E. Andert, H. Singh, A. Shrivastava, J. Laudon, Y. Zhou, B. Iannucci, and C. Joe-Wong, "Giph: Generalizable placement learning for adaptive heterogeneous computing," *Proceedings of Machine Learning and Systems*, vol. 5, 2023.
- [21] Y. Song, Z. Mi, H. Xie, and H. Chen, "Powerinfer: Fast large language model serving with a consumer-grade gpu," in *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, 2024, pp. 590–606.
- [22] K. Lu, Q. Wei, Y. Lin, P. Liu, H. Wang, J. Wan, T. Yao, H. Wu, and D. Wang, "Q-infer: Towards efficient gpu-cpu collaborative llm inference via sparsity-aware dynamic scheduling," *ACM Transactions on Architecture and Code Optimization*, vol. 22, no. 4, pp. 1–26, 2025.
- [23] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Ré, "Flashattention: Fast and memory-efficient exact attention with io-awareness," *Advances in neural information processing systems*, vol. 35, pp. 16 344–16 359, 2022.
- [24] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv *et al.*, "Qwen3 technical report," *arXiv preprint arXiv:2505.09388*, 2025.
- [25] H. Touvron, L. Martin, K. Stone *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [26] A. Grattafiori, A. Dubey, A. Jauhri *et al.*, "The llama 3 herd of models," *arXiv preprint arXiv:2407.21783*, 2024.
- [27] Mistral AI, "Mistral NeMo," <https://mistral.ai/news/mistral-nemo>, 2024.
- [28] OLMo Team, P. Walsh, L. Soldaini, D. Groeneveld *et al.*, "2 OLMo 2 furious," *arXiv preprint arXiv:2501.00656*, 2025.
- [29] Gemma Team, M. Riviere, S. Pathak *et al.*, "Gemma 2: Improving open language models at a practical size," *arXiv preprint arXiv:2408.00118*, 2024.



Atharva Khedkar Atharva Khedkar (Graduate Student Member, IEEE) received the B.Tech. degree in artificial intelligence from G. H. Raisoni College of Engineering, Nagpur, India, in 2023. He is currently pursuing the Ph.D. degree in computer science with Arizona State University, Tempe, AZ, USA. His research interests include compilers, machine-learning accelerators, and multi-level intermediate representations.



Shashwat Pandey Shashwat Pandey (Graduate Student Member, IEEE) received the B.Tech. degree in computer science from Christ University, Bengaluru, India, in 2022, and the M.S. degree in computer engineering from Arizona State University, Tempe, AZ, USA, in 2026. He is currently a Member of Technical Staff at AGI, Inc. His research interests include compilers, high-performance kernels, and large language model inference.



Aviral Shrivastava Aviral Shrivastava (Senior Member, IEEE) received the B.Tech. degree in computer science and engineering from the Indian Institute of Technology Delhi, New Delhi, India, in 1999, and the M.S. and Ph.D. degrees in information and computer science from the University of California, Irvine, CA, USA, in 2002 and 2006, respectively. He is currently a Professor with the School of Computing and Augmented Intelligence, Arizona State University, Tempe, AZ, USA, where he directs the Make Programming Simple Lab. His research interests include compilers, computer architecture, and machine-learning acceleration. He is the Editor-in-Chief of IEEE Embedded Systems Letters and a recipient of the NSF CAREER Award. He was an Associate Editor of this TRANSACTIONS from 2018 to 2023.